

Hospital Bed Allocation Platform

Where Linux Commands Are Used, and How Operating Systems Concepts Are Implemented

Course: Operating Systems (CCSE0303A) · B.Tech CSE-AI, Group 1

Team: Anant Pratap Singh, Aman Maurya, Akash Yadav, Abhijeet Chaudhary, Alok

This document explains, in plain language, exactly where real Linux commands run inside the project and why, and how each Operating Systems topic from the syllabus is actually implemented in the working application (not just mentioned in the report). It is meant as a quick reference for the viva.

1. Where Linux Commands Are Used, and Why

Every command below runs for real, using Python's **subprocess** module, on the “**Linux Monitor**” page of the app. Nothing is faked or simulated — when you open that page, the server actually executes these commands on the operating system it is running on and shows you the real output.

Command	What it does	Why it is used here
uname -a	Prints the operating system name, kernel version, and machine architecture.	Proves the app can query the real OS it is running on — the most basic OS-identification command.
whoami	Prints the username the server process is running as.	Shows which user account owns the running process.
pwd	Prints the current working directory of the server.	Shows where on disk the application is executing from.
ls -la	Lists all files and folders in the project directory.	Basic filesystem listing — the OS manages the filesystem, this reads it.
df -h	Shows how much disk space is used/free, in human-readable form.	Disk is one of the physical resources an OS manages — direct evidence of resource monitoring.
free -h	Shows RAM (memory) usage.	Memory is another OS-managed resource. (Linux-only command — on Mac/Windows the app shows “not available on this OS” instead of crashing.)
uptime	Shows how long the system has been running and its current load average.	Shows overall system health/load, a classic sysadmin command.
ps -e / ps aux	Lists every process currently running on the machine.	Direct, live proof that the OS is managing real processes — ties straight into the “Processes” syllabus topic.
ps aux wc -l	Counts how many processes are currently running.	Demonstrates a Linux pipe (): the output of one command feeds into another.

Why a fixed list of commands, not a free-text box

If the app let you type *any* Linux command into a web form and ran it directly, anyone could type something destructive (e.g. a delete command) and the server would obey — this is called **command injection**, a serious security flaw. Instead, this app only ever runs commands from a fixed, pre-approved list written directly in the Python code (**config.py**). No text typed by a user is ever inserted into a command. This is why the page only shows a fixed set of results rather than a command box.

2. Shell Scripts (the scripts/ folder)

Seven real Bash scripts automate setup and administration. They can be run from a terminal, or triggered with one click from the app's "OS Lab / System" page.

Script	What it does
setup.sh	Creates the Python virtual environment, installs required packages, and builds the database.
start_server.sh	Starts the Flask web server in the background and records its process ID.
stop_server.sh	Stops the server that start_server.sh started.
backup_database.sh	Makes a timestamped copy of the database file, in case something goes wrong.
system_report.sh	Runs several Linux commands together (OS info, CPU, memory, disk, process count, uptime) and saves them into one report file.
cleanup.sh	Deletes temporary Python cache files and old report files to keep the project tidy.
health_check.sh	Checks whether the running server is actually responding, and reports healthy/unhealthy.

3. How Each OS Topic Is Actually Implemented

The goal of this project was to make each Operating Systems concept something you can literally click and see happen, not just a paragraph in a report. Here is what each topic means in this project, and where to see it working.

OS Architecture

Every click on the website travels down a chain: Website (Browser) → Flask (the Python web app) → Python itself → the Operating System's interfaces → the Linux Kernel → the actual hardware (CPU / Memory / Disk). Nothing in this app skips that chain — even saving a patient's record ultimately becomes a real file-write request handled by the kernel.

Kernel & System Calls

The app's Python code never talks to the kernel directly. It uses built-in functions (**os.getpid()**, **open()**, **subprocess.run()**) that internally ask the operating system to do the work — get a process ID, open a file, start a new program. The Linux Monitor page shows these calls happening live.

Process Lifecycle

Every bed request submitted through the app is treated like an OS process and moves through real states: **NEW** → **READY** → **RUNNING** → **WAITING** (if no bed is free) → **TERMINATED** (once a bed is given). You can watch every request's current state on the "Process Monitor" page.

Process Control Block (PCB)

For every request, the app keeps one record holding everything a real OS would track about a process: an ID, its state, priority, when it arrived, how long it waited, and which bed it got. This is shown as a live table on Process Monitor. (It is a project-level model inspired by PCB theory, not the real Linux kernel's internal PCB.)

Process vs Thread

A dedicated page runs the exact same small task twice: once split across multiple **threads** (which share memory with each other) and once across multiple separate **processes** (which do not share memory). It times both and explains the difference in plain terms.

Multithreading

When several patients "request beds at the same time," the app genuinely starts multiple Python threads — this is not simulated with a delay or animation, real concurrent threads are created and run at once.

Synchronization / Race Conditions / Mutual Exclusion

This is demonstrated live, not just explained. On the Concurrency Demo page: turn the safety switch (a **lock**) OFF and run several patients against one bed — the app will actually show two patients wrongly getting the SAME bed (a genuine race condition). Turn the switch back ON and run it again — only one patient gets it and the rest correctly wait. This proves, rather than just claims, why locks are necessary.

CPU Scheduling (5 algorithms)

The app orders bed requests the same way an OS orders processes for the CPU:

- **FCFS** — whoever asked first is served first, regardless of urgency.
- **SJF** — the request needing the least processing time goes first.
- **SRTF** — like SJF, but can interrupt a request already in progress if a shorter one shows up.
- **Priority Scheduling** — emergency (Critical) patients are pushed ahead of routine ones.

- **Round Robin** — everyone gets a small, fixed turn, then goes to the back of the line if not finished, so nobody waits forever.

Scheduling Performance Metrics

Every algorithm run reports the four standard numbers taught in class: **Waiting Time**, **Turnaround Time**, **Response Time**, and **Throughput**. A separate page runs all 5 algorithms on the same request list and compares them side by side, with a chart.

Resource Management

Hospital beds are the limited, shared resource. Each bed is **Available**, **Allocated**, or under **Maintenance**. When a patient is discharged, their bed becomes available again and the app automatically checks whether a waiting patient can now be given it.

***Important academic note:** this project simulates these OS concepts at the application level to demonstrate understanding of them — it does not modify the real Linux kernel, its real scheduler, or its real process table. The Linux commands shown on the Linux Monitor page, however, are completely real and genuinely execute on the operating system, not simulated.*